



# Asistente de Auditoría

**Contraloría General de la República**

Documentación técnica de la solución

Arquitectura · Datos · ETL e ingesta · Auto Loader · Vector Search · Agente · Aplicación

Plataforma: Databricks (Unity Catalog, Mosaic AI, Model Serving, Apps)

**Daniel Vargas — Databricks**

# Contenido

---

1. Visión general y objetivos
2. Arquitectura de la solución
3. Modelo de datos (Unity Catalog)
4. ETL e ingesta dinámica (Auto Loader)
5. Vector Search (índice Delta Sync)
6. Agente de auditoría (Mosaic AI Agent Framework)
7. Generación del informe analítico (PDF)
8. Aplicación (Databricks App / Streamlit)
9. Despliegue (Asset Bundles) y prerequisites
10. Seguridad, permisos y trazabilidad
11. Flujo de una consulta de extremo a extremo
12. Apéndice: parámetros y archivos

# 1. Visión general y objetivos

La solución habilita dos usos complementarios sobre los informes de auditoría de la Contraloría, con foco en **planeación y focalización de auditorías** del sector Medio Ambiente (extensible a otros sectores):

- **Chat analítico:** preguntas y respuestas sobre los informes, con **citas a documento y página**, que distingue **hallazgos comprobados** de **líneas a verificar** y declara cuándo **no hay evidencia** suficiente. También redacta informes conversando en lenguaje natural (por sigla o nombre, con o sin tema, para uno o varios sujetos).
- **Informe analítico (PDF):** el mismo contenido en formato CGR descargable, con gráficas.

Además, al **cargar un PDF** se ingiere e indexa automáticamente, ampliando la base de conocimiento sin intervención manual.

**Principios de diseño:**

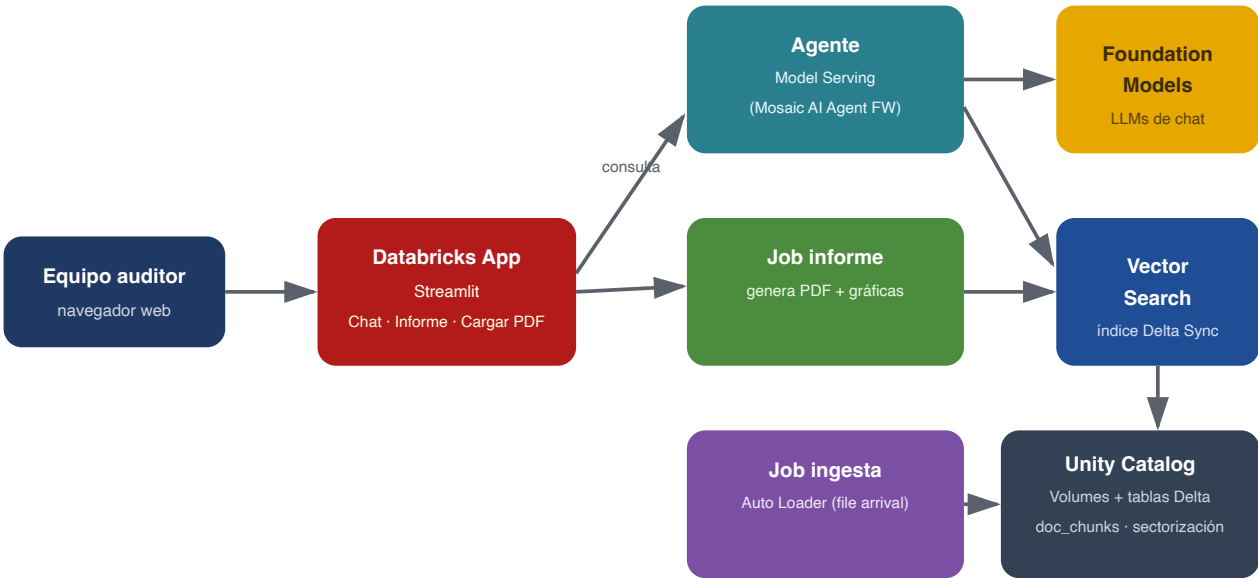
Citas verificablesComprobado ≠ hipótesisSin alucinaciones de cifrasIngesta dinámicaRéplica en pocos comandosLLMs autodetectados

## Capacidades que responde

| Necesidad del negocio                                                                  | Cómo la resuelve                                                                                                                  |
|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| ¿A qué sector/delegada pertenece un sujeto y qué otros sujetos comparten esa delegada? | Tabla <code>sectorizacion</code> + función UC <code>buscar_sectorizacion</code> (herramienta del agente).                         |
| Informes de auditoría y periodos cubiertos                                             | Metadata por chunk ( <code>documento</code> , <code>vigencia</code> , <code>tipo_auditoria</code> ) recuperada vía Vector Search. |
| Hallazgos, recurrencias e incidencia fiscal con cuantías                               | RAG sobre <code>doc_chunks</code> ; el prompt exige cita <code>[documento, p. N]</code> y cuantías exactas.                       |
| Riesgos para la próxima auditoría y qué revisar                                        | Sección "Líneas a verificar" separada de lo comprobado, con prueba sugerida.                                                      |
| Comparar varios sujetos                                                                | Análisis por sujeto + sección de riesgos y patrones compartidos.                                                                  |

## 2. Arquitectura de la solución

Todo se ejecuta dentro del workspace de Databricks del cliente, sobre Unity Catalog. Los datos nunca salen del perímetro de gobierno del cliente.



### Componentes

| Capa             | Servicio Databricks                              | Rol                                               |
|------------------|--------------------------------------------------|---------------------------------------------------|
| Presentación     | Databricks Apps (Streamlit)                      | Chat, generación de informes, carga de PDFs.      |
| Razonamiento     | Model Serving + Mosaic AI Agent Framework        | Agente con herramientas (RAG + función UC).       |
| Modelos          | Foundation Model APIs                            | LLMs de chat (autodetectados) y embeddings.       |
| Recuperación     | Mosaic AI Vector Search                          | Índice Delta Sync sobre <code>doc_chunks</code> . |
| Procesamiento    | Lakeflow Jobs + Spark (Auto Loader)              | Ingesta incremental, generación de informe.       |
| Datos y gobierno | Unity Catalog (Volumes, tablas Delta, funciones) | Almacenamiento gobernado y permisos.              |

### 3. Modelo de datos (Unity Catalog)

Todo vive bajo un `catalog.schema` configurable (por defecto `<catalog>.contraloria` ). Objetos:

#### 3.1 Volumes (almacenamiento de archivos)

| Volume                           | Contenido                         | Uso                                               |
|----------------------------------|-----------------------------------|---------------------------------------------------|
| <code>informes</code>            | PDFs fuente de auditoría (insumo) | Origen del Auto Loader; destino de "Cargar PDF".  |
| <code>informes-analiticos</code> | PDFs generados por el sistema     | Salida del job de informe; descarga desde la app. |

El Auto Loader mantiene su estado en `informes/_checkpoints/ingest` .

#### 3.2 Tabla `sectorizacion` + función UC

Relaciona cada sujeto de control con su **sigla**, **sector** y **Contraloría Delegada**. Se consulta mediante una función de Unity Catalog que el agente invoca como herramienta.

| Columna                                                                       | Descripción                                    |
|-------------------------------------------------------------------------------|------------------------------------------------|
| <code>sujeto</code>                                                           | Nombre completo del sujeto de control          |
| <code>sigla</code>                                                            | Sigla (CAR, CVC, PNNC, ANLA, FONDO_VIDA, MADS) |
| <code>sector</code> / <code>delegada</code>                                   | Sector y Contraloría Delegada                  |
| <code>delegado_titular</code> , <code>nit</code> , <code>observaciones</code> | Metadatos adicionales                          |

```
CREATE OR REPLACE FUNCTION buscar_sectorizacion(termino STRING)
RETURNS TABLE(sujeto, sigla, sector, delegada, delegado_titular)
RETURN SELECT ... FROM sectorizacion
    WHERE lower(sujeto) LIKE '%' || lower(termino) || '%'
       OR lower(sigla) LIKE '%' || lower(termino) || '%'
       OR lower(sector) LIKE '%' || lower(termino) || '%'
       OR lower(delegada) LIKE '%' || lower(termino) || '%';
```

La carga inicial (seed) se hace con un `MERGE` idempotente por `sigla` , de modo que re-ejecutar el setup no duplica filas.

#### 3.3 Tabla `doc_chunks` (base del RAG)

Un registro por **fragmento (chunk)** de PDF, con la metadata necesaria para citar la fuente. Tiene **Change Data Feed** activo para que Vector Search sincronice de forma incremental.

| Columna                                      | Tipo        | Descripción                                                 |
|----------------------------------------------|-------------|-------------------------------------------------------------|
| <code>chunk_id</code>                        | STRING (PK) | MD5 de <code>documento página índice</code> (idempotencia). |
| <code>documento</code>                       | STRING      | Nombre del PDF (se usa en la cita).                         |
| <code>path</code>                            | STRING      | Ruta completa en el Volume.                                 |
| <code>sigla, sujeto, sector, delegada</code> | STRING      | Denormalizados desde <code>sectorizacion</code> .           |
| <code>tipo_auditoria</code>                  | STRING      | Financiera / Cumplimiento / No determinado.                 |
| <code>vigencia</code>                        | STRING      | Año(s) detectados en el nombre del archivo.                 |
| <code>pagina</code>                          | INT         | Página del PDF (para la cita <code>[doc, p. N]</code> ).    |
| <code>chunk_index</code>                     | INT         | Orden del fragmento dentro de la página.                    |
| <code>texto</code>                           | STRING      | Texto del fragmento (columna que se vectoriza).             |

|             |           |                   |
|-------------|-----------|-------------------|
| ingested_at | TIMESTAMP | Marca de ingesta. |
|-------------|-----------|-------------------|

## 4. ETL e ingesta dinámica (Auto Loader)

El objetivo es que **dejar caer un PDF en el Volume** baste para que quede consultable. Se logra con un job disparado por **llegada de archivo** que corre Auto Loader de forma incremental.



### 4.1 Disparo por llegada de archivo

El job `ingest` define un *trigger* de tipo `file_arrival` sobre la URL del Volume. Databricks vigila el Volume y lanza el job cuando aparece un archivo nuevo.

```
trigger:
  file_arrival:
    url: /Volumes/${catalog}/${schema}/informes/
```

### 4.2 Lectura incremental (Auto Loader)

Se lee en modo `cloudFiles` con formato `binaryFile` (el PDF llega como bytes), filtrando `*.pdf`. El **checkpoint** garantiza que cada archivo se procese una sola vez; el trigger `availableNow` procesa lo pendiente y termina.

```
(spark.readStream.format("cloudFiles")
  .option("cloudFiles.format", "binaryFile")
  .option("pathGlobFilter", "*.pdf")
  .load(VOLUME_PATH)
  .writeStream.foreachBatch(upsert)
  .option("checkpointLocation", CHECKPOINT)
  .trigger(availableNow=True).start())
```

### 4.3 Parseo y troceado (chunking)

- **Por página:** `pypdf` extrae el texto de cada página; el número de página se conserva para la cita.
- **Chunks:** ventana de `CHUNK_SIZE = 1600` caracteres con `OVERLAP = 200` (solape que evita cortar ideas entre fragmentos).
- **Metadata derivada del nombre del archivo:**
  - `sigla`: por patrón (ANLA, PNNC, CVC, CAR, MADS, FONDO\_VIDA).
  - `tipo_auditoria`: Financiera / Cumplimiento.
  - `vigencia`: años `20\d\d`, tras remover el ID inicial del documento (p. ej. `202601005_`) para no confundirlo con la vigencia.

El resto (`sujeto`, `sector`, `delegada`) se toma de `sectorizacion`.

### 4.4 Escritura idempotente

Cada lote hace `MERGE INTO doc_chunks` por `chunk_id`: si el mismo PDF se re-sube, se actualizan los chunks en vez de duplicarlos. Al final del job se dispara `sync_index()` de Vector Search para refrescar el índice.

**Resultado:** subir un PDF (por la app o directo al Volume) ⇒ ingesta automática ⇒ chunks en `doc_chunks` ⇒ índice sincronizado ⇒ disponible en chat e informes, sin pasos manuales.

## 5. Vector Search (índice Delta Sync)

La recuperación semántica usa un índice **Delta Sync** gestionado, que se mantiene sincronizado con la tabla `doc_chunks` a través del Change Data Feed.

| Aspecto              | Configuración                                                |
|----------------------|--------------------------------------------------------------|
| Endpoint             | <code>contraloria_vs</code> (tipo STANDARD)                  |
| Índice               | <code>&lt;catalog&gt;.&lt;schema&gt;.doc_chunks_index</code> |
| Tabla origen         | <code>doc_chunks</code> (con Change Data Feed)               |
| Clave primaria       | <code>chunk_id</code>                                        |
| Columna vectorizada  | <code>texto</code>                                           |
| Modelo de embeddings | <code>databricks-gte-large-en</code>                         |
| Tipo de pipeline     | TRIGGERED (se sincroniza bajo demanda tras cada ingesta)     |

Las consultas devuelven, además del `texto`, los campos de metadata ( `documento`, `pagina`, `sigla`, `sujeto`, `sector`, `delegada`, `tipo_auditoria`, `vigencia` ) que permiten **citar** y **filtrar** (p. ej. por `sigla` en la generación de informe).

## 6. Agente de auditoría (Mosaic AI Agent Framework)

Es un único agente implementado como `ResponsesAgent` de MLflow y desplegado en Model Serving. Ejecuta un **bucle de tool-calling** sobre un LLM de chat ( `ChatDatabricks` + `bind_tools` ), sin dependencias de orquestación pesadas.

### 6.1 Herramientas

| Herramienta                          | Qué hace                                                                                                                                                                 |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>retrieve_audit_evidence</code> | RAG sobre Vector Search; devuelve fragmentos con <code>documento</code> y <code>pagina</code> para citar. Es la fuente de hallazgos, cuantías, opiniones y antecedentes. |
| <code>buscar_sectorizacion</code>    | Función UC: resuelve sigla↔nombre, sector, delegada y sujetos asociados.                                                                                                 |

### 6.2 Reglas del sistema (system prompt)

- Reconoce al sujeto por **sigla o nombre completo**.
- Toda afirmación basada en informes cita `[documento, p. N]`.
- Separa **Hallazgos comprobados** (con cita y cuantía) de **Líneas a verificar** (hipótesis a confirmar por el equipo auditor).
- Si no hay evidencia, lo declara y **no inventa** cifras, páginas ni documentos.
- Para "informe", estructura por sujeto: antecedentes, hallazgos + cuantías, planes de mejoramiento, recurrencias/persistencia, riesgos/líneas a verificar y conclusión técnica.

### 6.3 Una consulta = varias llamadas

Cada turno del usuario puede implicar **varias llamadas** al LLM: el modelo decide llamar herramientas (búsquedas en Vector Search y/o sectorización), recibe los resultados y vuelve a razonar, hasta producir la respuesta final (tope `MAX_ITERS = 14`). Cada consulta a Vector Search es, a su vez, una llamada al modelo de embeddings.

### 6.4 Selección dinámica de modelo y streaming

- **LLM por petición**: el cliente envía `custom_inputs.llm_endpoint` y el agente usa ese modelo (cachea un `ChatDatabricks` por endpoint). Para modelos GPT de razonamiento añade `reasoning_effort="none"` para habilitar tool-calling.
- **Streaming** ( `predict_stream` ): emite la respuesta final token a token. Se **descarta** el texto de los turnos que llaman herramientas y los bloques de "razonamiento" de los modelos con *thinking*, para que no se filtren al usuario.

### 6.5 Autodetección de LLMs

Al desplegar, se listan los serving endpoints de chat ( `task = llm/v1/chat` ) de la cuenta y se conceden al agente los **modelos recomendados que existan** (con *fallback* a los primeros detectados). Así la solución se adapta a los modelos de cada workspace sin edición manual.

**Recursos del modelo (auth passthrough)**: endpoints de LLM + endpoint de embeddings + índice de Vector Search + función `buscar_sectorizacion`. El endpoint del agente obtiene permiso para invocarlos.

## 7. Generación del informe analítico (PDF)

---

Un job dedicado produce el informe en formato CGR. Lo dispara la app (o se corre con parámetros).

1. **Resolución de sujetos:** acepta siglas o nombres y los normaliza a siglas.
2. **Recuperación en paralelo:** por cada sujeto y cada tema núcleo se consulta Vector Search ( `ThreadPoolExecutor` ), con tope de fragmentos por sujeto para acotar el prompt. Si se indica un **tema/enfoque**, las consultas se focalizan en él.
3. **Dos llamadas al LLM:** (a) redacta el informe en Markdown (estructura CGR, hasta 16k tokens); (b) extrae **solo datos numéricos** en JSON para graficar (sin inventar).
4. **Gráficas:** matplotlib genera hasta cuatro visualizaciones coloridas (hallazgos por informe, concentración por sujeto, cuantías fiscales, calificación de control interno con bandas).
5. **Ensamblado PDF:** el texto se renderiza con xhtml2pdf (encabezados, tablas, pie CGR) y se **fusiona** con la página de gráficas usando pypdf; el resultado se guarda en el Volume `informes-analiticos` .

El mismo contenido se puede pedir conversando en el chat; el job es la vía para obtener el PDF con gráficas.

## 8. Aplicación (Databricks App / Streamlit)

Interfaz institucional (tema oscuro, escudo y bandera de Colombia) con tres páginas y un selector de modelo global en la barra lateral.

| Página            | Función                                                                                                                                                                                                                              |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Chat              | Conversa con el agente. Respuesta en <b>streaming</b> con indicador "Consultando informes"; <b>memoria</b> de conversación (se reenvía el historial en cada turno); botón "Nueva conversación"; descarga de cada respuesta como PDF. |
| Informe analítico | Formulario (sector, sujetos, periodo, tema) que dispara el job y permite descargar el PDF cuando termina.                                                                                                                            |
| Cargar PDF        | Sube un informe al Volume <b>informes</b> ; la ingesta e indexación se disparan solas.                                                                                                                                               |

### 8.1 Detalles de implementación

- **Selector de LLM dinámico:** lista los modelos de chat disponibles (recomendados que existan), con lista curada de respaldo.
- **Streaming SSE:** lee los eventos `response.output_text.delta` del endpoint del agente; hace *fallback* a modo no-streaming si no está disponible.
- **Montos en pesos:** se escapan los `$` al renderizar para que Streamlit no los interprete como fórmulas LaTeX.
- **Memoria:** es a nivel de sesión; al reenviar el historial completo, el agente mantiene el hilo.

## 9. Despliegue (Asset Bundles) y prerequisites

Toda la solución se empaqueta como un **Databricks Asset Bundle**. Un solo script la despliega de extremo a extremo.

```
./deploy.sh <perfil-cli>
# bundle deploy -> setup -> ingest -> deploy_agent (autodetecta LLMs)
#                               -> publica la app -> concede al SP de la app acceso a los Volumes
```

### 9.1 Qué editar antes de desplegar

| En <code>databricks.yml</code>             | Valor                         |
|--------------------------------------------|-------------------------------|
| <code>workspace.host</code>                | URL del workspace del cliente |
| <code>catalog</code> / <code>schema</code> | Dónde viven los datos         |
| <code>warehouse_id</code>                  | SQL Warehouse serverless      |

### 9.2 Prerrequisitos

- Unity Catalog habilitado (catalog + schema para la solución).
- SQL Warehouse serverless; Vector Search habilitado; Databricks Apps habilitado.
- Foundation Model APIs activas: al menos un modelo de chat y `databricks-gte-large-en`.
- Rol de quien despliega: crear tablas/funciones/Volumes, registrar modelos en UC, crear serving endpoints, Vector Search endpoints, jobs y apps; MANAGE/OWNER sobre los securables para los grants.

## 10. Seguridad, permisos y trazabilidad

---

- **Datos en el perímetro del cliente:** PDFs, chunks y vectores viven en Unity Catalog del workspace; no se exportan a terceros.
- **Service principals:** la app y el endpoint del agente corren con SP propios; se les concede lo mínimo (la app: `USE CATALOG/SCHEMA` + `READ/WRITE VOLUME`; el agente: invocar LLMs, embeddings, índice y función).
- **Gobierno UC:** los permisos sobre tablas, funciones y Volumes se administran con Unity Catalog.
- **Trazabilidad:** MLflow Tracing en el agente y la tabla de *payload/inferencia* del endpoint permiten auditar cada conversación (entradas, herramientas usadas, salida).
- **Antialucinación:** el prompt prohíbe inventar cifras/páginas/documentos y exige citar la fuente; las cuantías se reportan tal como aparecen.

## 11. Flujo de una consulta de extremo a extremo

---

1. El usuario escribe en el **Chat** (elige modelo en la barra lateral).
2. La app envía el historial al **endpoint del agente** con `stream: true` y `custom_inputs.llm_endpoint`.
3. El agente razona y, si hace falta, llama `buscar_sectorizacion` y/o `retrieve_audit_evidence` (Vector Search + embeddings).
4. Con la evidencia, redacta la respuesta citando `[documento, p. N]` y separando comprobado de líneas a verificar.
5. La app muestra la respuesta en streaming y ofrece descargarla en PDF.

## 12. Apéndice: parámetros y archivos

### Parámetros configurables (bundle)

| Variable           | Por defecto                | Descripción                 |
|--------------------|----------------------------|-----------------------------|
| catalog / schema   | —/ contraloria             | Ubicación de los datos.     |
| volume_informes    | informes                   | PDFs fuente.                |
| volume_analiticos  | informes-analiticos        | PDFs generados.             |
| vs_endpoint        | contraloria_vs             | Endpoint de Vector Search.  |
| embedding_endpoint | databricks-gte-large-en    | Embeddings.                 |
| llm_endpoint       | databricks-claude-opus-4-7 | LLM por defecto del agente. |
| agent_endpoint     | contraloria_audit_agent    | Endpoint del agente.        |
| warehouse_id       | (editar)                   | SQL Warehouse serverless.   |

### Estructura del repositorio

| Archivo                | Rol                                                        |
|------------------------|------------------------------------------------------------|
| deploy.sh              | Despliegue completo en un comando (incluye grants).        |
| databricks.yml         | Bundle: variables y target.                                |
| resources/jobs.yml     | Jobs: setup, ingest, deploy_agent, generate_report.        |
| resources/app.yml      | Recurso de la app y sus permisos.                          |
| src/setup.py           | Sectorización + función UC + tabla chunks + Vector Search. |
| src/ingest.py          | Auto Loader: PDF → chunks → MERGE → sync índice.           |
| src/agent.py           | Definición del agente (código del modelo).                 |
| src/deploy_agent.py    | Registra y despliega el agente (autodetecta LLMs).         |
| src/generate_report.py | Genera el informe analítico en PDF.                        |
| app/                   | App Streamlit (app.py, app.yaml, requirements, assets).    |